EPHEC
ÉCOLE SUPÉRIEURE
ENSEIGNEMENT POUR ADULTES

Année académique **2026-2027**

Support de cours - Programmation orientée objet - Java

Module INI

Introduction au cours

Ce que couvre le cours, comment il s'organise, et comment travailler

EPHEC — Bachelier en informatique, niveau 2

Cette introduction présente le cours : ce qu'il couvre et ce que vous devrez savoir faire au terme de l'année, comment il s'organise et comment il est évalué, puis la façon dont s'articulent les différents supports, et les documents de référence qui vous accompagneront toute l'année.

INI §1 — Le cours

Qui suis-je ?

Avant d'entrer dans le cours, quelques mots sur celui qui le donne.

Mon parcours professionnel a commencé au helpdesk d'ING, qui s'appelait encore la BBL à l'époque. Je suis ensuite devenu testeur, puis développeur : c'est à ce moment-là que j'ai suivi, à l'EPHEC, le même parcours que vous. Je suis devenu ensuite analyste fonctionnel, puis Scrum Master, et j'évolue aujourd'hui vers un rôle de coach, d'équipe ou d'organisation.

Ce qui m'a d'abord attiré vers l'informatique, c'est sa capacité à rendre visibles certains objets mathématiques, comme l'ensemble de Mandelbrot ou le jeu de la vie de Conway.

J'aime les années 80, qui ont bercé mon adolescence : Simple Minds, Depeche Mode, le film *Blade Runner*. J'aime aussi le café.

Enfin, je suis attentif, sans doute davantage depuis que j'ai trois enfants, aux problématiques systémiques dans lesquelles notre monde est plongé : les ressources, l'énergie, la biodiversité. C'est ce qui m'a poussé à m'inscrire à une formation en développement durable.

Contenu du cours

Ce cours approfondit et met en pratique ce que vous avez vu en C#. Il aborde des concepts plus avancés de la programmation orientée objet, comme l'héritage, les types génériques et les expressions lambda, quelques classes de la bibliothèque standard de Java, et quelques outils, dont `JUnit` pour les tests et `JavaFX` pour les interfaces graphiques.

Chaque notion est expliquée, puis pratiquée au travers d'exercices. Le travail personnel entre les séances est indispensable : c'est lui qui permet d'assimiler ce qui a été vu au cours. Si vous rencontrez une difficulté, signalez-la au plus vite plutôt que de la laisser s'installer.

Les modules du cours

Le cours est découpé en modules. Chacun porte un code de trois lettres, que vous retrouverez partout : sur les slides, dans le nom des exercices, dans les packages Java et sur Moodle. Le code désigne le sujet, pas la séance : un module peut occuper moins d'une séance, ou en déborder.

Code	Sujet
`POO`	Programmation orientée objet : classes et objets
`TMP`	Temps : dates et durées
`TAB`	Tableaux
`ENU`	Énumérations et records
`ABS`	Héritage, classes abstraites
`ITF`	Interfaces
`PKG`	Packages et visibilité
`COL`	Collections
`EXC`	Exceptions

Code	Sujet
JUN	Tests unitaires (JUnit)
GEN	Types génériques
LAM	Classes anonymes et expressions lambda
FX	Interfaces graphiques (JavaFX)
FIC	Fichiers
REC	Exercices récapitulatifs

L'ordre est indicatif. Il peut évoluer en cours d'année ; Moodle indique, au fil des séances, les modules abordés.

Les acquis d'apprentissage

Les acquis d'apprentissage, repris du dossier pédagogique du cours, décrivent ce que vous devez être capable de faire au terme du cours. Ils se regroupent en deux familles.

La première porte sur les objets : l'étudiant sera capable de concevoir, d'installer et d'utiliser des objets appropriés à la solution. Concrètement, il pourra :

- définir et implémenter des classes et des objets Java simples ;
- compléter des classes Java implémentées par un tiers ;
- utiliser des classes et des objets Java existants, et interagir avec eux ;
- dériver de nouvelles classes par héritage d'une classe Java existante ;
- utiliser les classes du JDK pour accéder à des fichiers, et pour réaliser une application disposant d'une interface graphique simple (JavaFX) ;
- démontrer, au travers d'un exercice simple, sa compréhension du concept et du fonctionnement des expressions lambda.

La seconde porte sur les tests : l'étudiant sera capable de concevoir et de mettre en œuvre une procédure de test partiel et intégré. Il pourra :

- implémenter des classes de test simples (JUnit), à partir d'une classe implémentée par un tiers ou de spécifications techniques ;
- implémenter des classes et des objets Java simples, et les classes de test correspondantes ;
- repérer et corriger des problèmes simples dans des classes écrites par un tiers, en exécutant une batterie de tests JUnit.

En d'autres termes, vous devez pouvoir concevoir des classes et les utiliser, employer l'héritage à bon escient, écrire et exécuter des tests qui vérifient qu'une classe fonctionne, et utiliser les classes du JDK vues au cours : fichiers, collections, exceptions et événements. Les deux premiers points prolongent ce que vous connaissez déjà en C#.

L'organisation pratique

Le cours compte quatorze séances, du 19 janvier au 18 mai 2027. Chaque séance commence à 18 h 00 et se termine au plus tard à 21 h 35, avec une pause de quinze minutes, et alterne théorie, démonstration et exercices.

Vous travaillerez avec Java et l'environnement de développement IntelliJ IDEA. Maven gère la structure des projets et les bibliothèques, JUnit sert aux tests et JavaFX aux interfaces graphiques. Les versions à utiliser sont annoncées sur Moodle : travaillez avec celles-là toute l'année, ce sont celles de l'examen.

> **Attention** - Présence aux séances
>
> Message officiel de la direction de l'EPHEC : « En cas de dépassement des 40 % d'absences injustifiées dans une unité d'enseignement, l'étudiant se verra automatiquement désinscrit du cours et non-admis à l'examen. »

L'examen

L'examen a lieu le 8 juin 2027. Il se déroule à cahier ouvert : il ne demande pas de par cœur, mais la capacité à écrire du code qui fonctionne. Il se passe sur machine et dure deux heures trente.

Il compte cinq ou six questions, dont la forme reste stable d'une session à l'autre :

- écrire une classe simple, qui lève une exception ;
- écrire un type générique ;
- écrire et exécuter une classe de tests pour une classe donnée ;
- écrire une hiérarchie d'interfaces et de classes ;
- compléter une application graphique pour qu'elle fonctionne ;
- et peut-être une sixième question, dont l'existence et le sujet vous seront confirmés en cours d'année.

Chaque question correspond à un ou plusieurs modules. Les exercices récapitulatifs de fin d'année en reprennent la structure.

INI §2 — Ma façon de travailler

Le déroulé d'un module

Un module suit toujours le même chemin :

Étape	Ce qu'elle apporte
Les slides	les notions, présentées en séance
La démonstration	les notions mises en pratique devant vous, en direct dans IntelliJ
Les exercices	votre pratique, à partir du fascicule d'exercices du module
GameHub	le fil rouge, un projet plus grand que vous faites évoluer à chaque séance

Les slides ne contiennent pas les énoncés : une slide de renvoi indique le code des exercices à faire, par exemple `POO.1`, et l'énoncé complet se trouve dans le fascicule d'exercices du module.

Les fascicules d'exercices

Chaque module a son fascicule d'exercices, distribué en PDF ; le recueil des exercices les rassemble en un seul document. Les exercices y sont numérotés par sujet : `TAB.3` est le troisième exercice sur les tableaux, quelle que soit la séance où on le fait. Le même code donne le nom du package dans le projet IntelliJ : `TAB.3` se trouve dans `be.ephec.java.tab.e03`.

Tous les énoncés suivent la même structure : l'objectif, le contexte, l'énoncé, les spécifications à respecter, la manière de vérifier votre code, et les questions à se poser. Cette dernière rubrique est la plus importante : elle signale ce que l'exercice cherche à vous faire découvrir.

Le projet ExercicesJava

Les exercices sont livrés dans un projet Maven, ExercicesJava, à ouvrir dans IntelliJ. À chaque séance, une archive zip ajoute les exercices du jour ; les solutions sont distribuées après le cours. Le guide IntelliJ et le mémento technique expliquent comment ouvrir le projet.

Les classes sont déjà écrites, avec leurs méthodes. Le corps de chaque méthode se contente de lever une `UnsupportedOperationException`, qui signifie « pas encore écrit » :

```
public int computeAge(int annee) {
    throw new UnsupportedOperationException(
            "computeAge n'est pas encore implementee");
}
```

Le projet compile donc dès le départ : c'est à vous de remplacer ces lignes par le vrai code.

À l'examen, en revanche, certaines questions vous demanderont de créer vous-même une classe qui n'existe pas encore : le fichier, la déclaration de la classe et ses méthodes (voir `IDE.8` du guide IntelliJ). Profitez des exercices pour vous y entraîner, en écrivant parfois une classe de toutes pièces plutôt qu'en complétant le squelette fourni.

Les tests fournis

La plupart des exercices sont accompagnés d'une classe de test `JUnit`. Vous la lancez vous-même, aussi souvent que vous le voulez : c'est votre moyen de savoir si votre réponse est correcte, sans attendre la séance suivante. Un test rouge signale que la réponse n'est pas encore correcte ; quand tous les tests sont verts, elle l'est.

Un test vert ne dit pas que votre code est bien écrit, seulement qu'il fait ce qui est demandé. Et les tests ne vérifient jamais un point que l'énoncé laisse à votre choix.

Vous n'avez pas besoin de connaître `JUnit` pour lancer ces tests : le mémento Exercices explique comment faire, et le module `JUN` vous apprendra à écrire les vôtres.

Le fil rouge GameHub

GameHub est une plateforme de jeux rudimentaire, construite petit à petit au fil de l'année : chaque module y ajoute une pièce, qui s'appuie sur ce qui a été construit précédemment. Ses énoncés sont numérotés `G.1`, `G.2`, et ainsi de suite, dans l'ordre imposé par leurs dépendances. GameHub vit dans un projet séparé des exercices.

Là où un exercice isole une notion, GameHub vous oblige à la faire cohabiter avec tout ce que vous avez déjà écrit. C'est la situation que vous retrouverez à l'examen, et surtout dans tout projet réel.

Ce support de cours

Les slides accompagnent l'explication orale. Ce support reprend chaque notion en prose, avec le code formaté, module par module. C'est lui qu'il faut relire entre les séances.

INI §3 — Les documents de référence

Quatre documents

À côté des fascicules et de ce support, quatre documents de référence vous accompagnent toute l'année. On ne les lit pas d'une traite, à part le mémento Exercices : on les ouvre quand un énoncé ou une slide y renvoie. Vous les trouverez en annexe de ce support, et sur Moodle, dans la section Documents.

Document	Préfixe	À quoi il sert
Mémento Exercices		lancer les tests, lire un résultat
Guide IntelliJ	`IDE`, `ERR`	se servir de l'outil, comprendre une erreur
Mémento technique	`TEC`	comprendre ce qui se passe sous le capot
Mémento Clean Code	`CC`	écrire du code lisible

Le **mémento Exercices** rassemble ce qui vaut pour tous les fascicules, et qu'il serait inutile de répéter dans chacun. Lisez-le une fois en début d'année. Il explique la numérotation des exercices, montre comment lancer les classes de test fournies et lire leur résultat, et pose une règle :

> **À retenir** - gardez toujours un code qui compile
>
> Tout le dossier `src/main/java` se compile d'un seul bloc : un exercice laissé à moitié écrit empêche de lancer tous les autres, y compris ceux des séances précédentes. Avant de passer à autre chose, vérifiez que votre code compile. Sinon, deux issues : le corriger, ou mettre le morceau en chantier entre commentaires avec `Ctrl+/` (`⌘/` sur Mac). C'est aussi ce que l'examen demande : on y rend du code qui compile.

Le **guide IntelliJ** couvre ce qu'il faut savoir d'IntelliJ pour ce cours, captures d'écran à l'appui : l'installation, la création et l'ouverture d'un projet, l'exécution, le débogage, le paramétrage. Ses entrées `ERR` partent du message affiché et expliquent sa cause la plus probable : c'est le premier réflexe à avoir quand quelque chose ne fonctionne pas.

Le **mémento technique** explique ce qu'IntelliJ fait à votre place : ce que sont la JVM, le JRE et le JDK, comment un fichier `.java` devient un programme, comment Maven récupère les bibliothèques, comment se lance une application `JavaFX`. Vous pouvez écrire vos premiers programmes sans le lire, mais il répond à la plupart des « pourquoi ça marche ? ».

Le **mémento Clean Code** rassemble 37 règles simples, en sept sections (noms, méthodes, commentaires, objets, erreurs, tests, classes), pour écrire du code qu'un autre développeur, ou vous-même dans six mois, comprendra sans effort. Un programme qui fonctionne n'est pas forcément un programme lisible. Ces règles s'inspirent du livre *Clean Code* de Robert C. Martin. Elles ne font pas l'objet d'un module : elles arrivent par petites touches, au fil des exercices, quand elles deviennent utiles. Elles s'appliquent aussi à l'examen : un code qui fonctionne mais ne les respecte pas peut y perdre des points.

Comment lire un renvoi

Les fascicules, les slides et ce support renvoient aux mémentos par des codes :

Renvoi	Où chercher
`IDE.14`	la section 14 du guide IntelliJ
`CC-MET.8`	la règle 8 de la section « méthodes » du mémento Clean Code
`TEC-EXE.4`	la section 4 de la partie « exécution » du mémento technique
`POO §6`	la section 6 du module `POO`, dans ce support

Un renvoi vous dit où chercher, au moment où vous en avez besoin. Par exemple, l'exercice `POO.5` vous demande de lire une Javadoc : il vous renvoie à `IDE.14`, qui montre comment l'afficher sans quitter IntelliJ. Le premier renvoi arrive dès le module `POO` : prenez l'habitude de le suivre.

Quelques liens utiles

La documentation officielle de Java, et notamment la Javadoc qui décrit chaque classe de la bibliothèque standard, se trouve sur le site d'Oracle : `docs.oracle.com/en/java/javase/`. Choisissez la version de Java annoncée sur Moodle : les classes évoluent d'une version à l'autre.

Le site de Jean-Michel Doudoux, `jmdoudoux.fr/java/dej/`, propose en français un tour très complet du langage et de sa bibliothèque.

En résumé

- Un module, c'est un sujet, avec son code de trois lettres.
- Chaque module suit le même chemin : slides, démonstration, exercices du fascicule, GameHub.
- Des tests vous permettent de vérifier vous-même vos réponses.
- Les mémentos s'ouvrent quand un renvoi vous y envoie.

Le module suivant, `POO`, entre dans le vif du sujet : il commence par ce qui change quand on passe de C# à Java.